

Exercícios de Revisão

1. Pilares da POO

Relacione cada situação ao conceito mais adequado: **abstração, encapsulamento, herança ou polimorfismo.**

- a. Uma classe representa apenas as informações relevantes de um funcionário para determinado sistema.
- b. Uma classe mantém seu saldo como private e controla as alterações por meio de métodos.
- c. Uma classe Carro herda de Veiculo .
- d. O método EmitirSom() produz resultados diferentes para um cachorro e para um gato.

Resposta:

- a → abstração
- b → encapsulamento
- c → herança
- d → polimorfismo

2. Encapsulamento

Analise:

```
public class Conta
{
    public double saldo;
}
```

- a) Cite dois problemas que podem ocorrer com essa implementação.

1º O atributo saldo é do tipo double, que não é o mais adequado para representar valores monetários; decimal é mais apropriado.

2º O atributo saldo é public, permitindo que qualquer parte do código altere seu valor diretamente, inclusive para valores inválidos, como um saldo negativo.

- b) Explique como sua solução controla o acesso ao saldo.

O saldo deve ser privado, impedindo alterações diretas por código externo. As alterações devem ser realizadas por métodos, como Depositar() e Sacar(), que podem aplicar validações antes de modificar o saldo.

3. Get e Set

Considere:

```
private string nome;

public void SetNome(string nome)
{
    this.nome = nome;
}
public string GetNome()
{
    return nome;
}
```

a) Qual método altera o atributo?

O método que altera o atributo é o SetNome.

b) Qual método retorna o atributo?

O método que retorna o atributo é o GetNome.

c) Explique a função de `this.nome = nome` .

`this.` serve para referenciar o atributo e diferenciá-lo da variável de mesmo nome.

4. Propriedades

Analise:

```
public class Produto
{
    private double preco;
    public double Preco
    {
        get { return preco; }
        set
        {
            if (value >= 0)
                preco = value;
        }
    }
}
```

a) Qual é a função do get ?

A função do get é retornar o valor do atributo preço.

b) Qual é a função do set ?

A função do set é validar e fazer a alteração do valor presente no atributo preço.

c) O que representa value ?

value representa o valor que o programa enviou para a classe para ser o novo valor do atributo preço.

d) O que acontece ao executar `produto.Preco = -10;` ?

Não ocorre erro de compilação. O valor -10 é enviado para value, mas como ele não atende à condição `value >= 0`, o atributo `preco` não é alterado. O programa continua normalmente.

5. Herança ou composição?

Classifique cada relação como herança ou composição e justifique usando "é um" ou "tem um".

- a. Carro — Veículo
- b. Carro — Motor
- c. Professor — Pessoa
- d. Pedido — Produto
- e. Gato — Animal

Resposta:

- a. Carro é um Veículo
- b. Carro tem um Motor.
- c. Professor é uma Pessoa.
- d. Pedido tem um Produto.
- e. Gato é um Animal.

6. Polimorfismo

Considere:

```
public class Animal
{
    public virtual void EmitirSom()
    {
        Console.WriteLine("Som");
    }
}
public class Cachorro : Animal
{
    public override void EmitirSom()
    {
        Console.WriteLine("Au au");
    }
}
```

a) Por que o método da classe Animal utiliza virtual ?

O método da classe Animal utiliza virtual pois o virtual informa que uma classe derivada poderá substituir o comportamento desse método.

b) Por que Cachorro utiliza override ?

Cachorro utiliza override pois override indica que a classe derivada está substituindo a implementação herdada da classe base.

c) Qual conceito de POO está representado?

O conceito de Polimorfismo.

7. Classe abstrata

Analise:

```
public abstract class Veiculo
{
    public abstract void LigarMotor();
    public void Parar()
    {
        Console.WriteLine("Parado");
    }
}
```

a) A classe pode ser instanciada diretamente? Explique.

Não. Uma classe abstrata não pode ser instanciada diretamente e serve como base para outras classes.

b) A classe pode possuir métodos com implementação?

Sim.

c) O que uma classe derivada precisa fazer com LigarMotor() ?

A classe derivada deve implementar o método LigarMotor(), pois ele foi declarado como abstract na classe base.

8. Interface

Explique por que a interface pode ser entendida como um contrato.

Uma interface pode ser entendida como um contrato porque define quais métodos uma classe deve implementar. A classe que implementa a interface se compromete a fornecer esses métodos, mas pode definir livremente como eles serão implementados.

9. Classe abstrata × interface

Explique duas diferenças entre uma classe abstrata e uma interface.

1. Uma classe abstrata pode possuir métodos concretos e métodos abstratos, enquanto uma interface é utilizada principalmente para definir um contrato que as classes devem implementar.
2. Uma classe pode herdar de apenas uma classe, mas pode implementar várias interfaces.

Depois indique qual conceito representa melhor cada frase:

- a. "É uma classe base que não pode ser instanciada diretamente."
- b. "Define um contrato que deve ser implementado."
- c. "Uma classe pode implementar mais de uma."
- d. "Pode possuir métodos concretos e métodos abstratos."

Resposta:

interface corresponde as letras b e c.

classe abstrata corresponde as letras a e d.

10. Valor ou referência?

Determine o resultado final:

```
int a = 10;  
int b = a;  
b = 20;
```

```
Pessoa p1 = new Pessoa();  
p1.Nome = "João";
```

```
Pessoa p2 = p1;  
p2.Nome = "Carlos";
```

a) Qual será o valor de a ?

O valor de a continuará sendo 10.

b) Qual será o valor de p1.Nome ?

O valor de p1.Nome será Carlos.

c) Explique a diferença entre os dois comportamentos.

Isso acontece porque int é um tipo por valor: quando b = a, o valor de a é copiado para b, criando valores independentes. Já Pessoa é uma classe, portanto é um tipo por referência: p1 e p2 passam a referenciar o mesmo objeto na memória. Assim, uma alteração feita por p2 também pode ser observada por p1.

11. Conversões

Classifique cada conversão como implícita, explícita (casting) ou por método.

- a. double y = 10;
- b. int x = (int)9.8;
- c. int numero = int.Parse("123");
- d. double valor = 15;
- e. int numero = (int)20.5;

Resposta:

- a → implícita
- b → explícita
- c → por método

d → implícita

e → explícita

12. Questão de raciocínio

Analise o código:

```
public class Pessoa
{
    private string nome;
    public string Nome
    {
        get { return nome; }
        set
        {
            if (value != "")
                nome = value;
        }
    }
}

Pessoa p = new Pessoa();
p.Nome = "João";
p.Nome = "";
Console.WriteLine(p.Nome);
```

a) O que será exibido?

Será exibido o nome João.

b) Explique passo a passo o que acontece nas duas atribuições.

Na primeira atribuição p.Nome envia João para a propriedade e ele passa pela verificação do set, sendo assim p.Nome = João, já na segunda atribuição é passado para a propriedade uma string em branco, ao chegar no set ela não passa na verificação, assim sendo p.Nome continua sendo João.

c) Qual conceito de POO está sendo utilizado para controlar o acesso ao atributo?

Está sendo utilizado o conceito de Encapsulamento

13. Análise completa

Considere:

```
public class Veiculo
{
    protected string modelo;
    public virtual void Acelerar()
    {
        Console.WriteLine("Veículo acelerando");
    }
}

public class Carro : Veiculo
{
    private Motor motor;
    public override void Acelerar()
    {
        Console.WriteLine("Carro acelerando");
    }
}
```

Identifique no código:

a. um exemplo de herança;

public class Carro : Veiculo , é um exemplo de herança pois a classe carro está herdando da classe Veiculo.

b. um exemplo de composição;

private Motor motor; é um exemplo de composição, pois a classe Carro possui uma referência para um objeto Motor.

c. um exemplo de polimorfismo;

```
public virtual void Acelerar()
{
    Console.WriteLine("Veículo acelerando");
} // na classe Veiculo
```

```
public override void Acelerar()
{
    Console.WriteLine("Carro acelerando");
} // na classe Carro é um exemplo de polimorfismo.
```